Charged Particle around a Dipole Simulation
Computational Science

Tom Erez
Under the Guidance of:
Shlomo Rozenfield

2026

A study on the numerical stability and deterministic chaos of charged particles within a static electrostatic dipole field, implemented via Velocity Verlet integration.

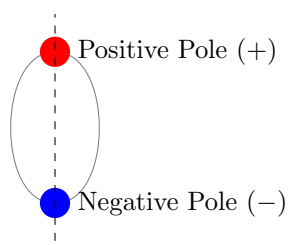


Figure 1: Schematic of the electrostatic dipole system.

Contents

1 Abstract **3**

2 Introduction **4**

3 Theoretical Basis **5**
3.1 Governing Physical Laws 5
3.1.1 Particle Movement 5
3.1.2 Conservation of Energy 5
3.2 Numerical Methods 6
3.2.1 Mathematical Theory 6
3.2.2 Euler's Method 6
3.2.3 Verlet's Method 7
3.2.4 Conservation of Energy in Numerical Methods 7
3.3 Chaos Theory 7

4 Methodology **8**
4.1 Development Environment and Software Stack 8
4.2 System Configuration 8
4.2.1 The Electrostatic Dipole Environment 8
4.2.2 Initial Particle Properties 8
4.3 Code Structure and Implementation 8
4.3.1 Particle Class 8
4.3.2 Acceleration Calculation 9
4.3.3 Implementation of Euler's Method 9
4.3.4 Implementation of Velocity Verlet Integration Method 10
4.3.5 Parameter Choices 11
4.4 Simulating Trajectories of Particles 11
4.4.1 Simulations of Cases 11
4.4.2 Comparison of Energy Conservation between Numerical Methods 11
4.4.3 Investigating Chaotic Properties of the System 11

5 Results **12**
5.1 Trajectory Simulations 12
5.1.1 Stable Orbit 12
5.1.2 Sling Shot 13
5.1.3 Scattering Movement 14
5.1.4 Circular Motion 15
5.2 Energy Conservation of Numerical Methods 16
5.3 Chaotic Properties 17
5.3.1 Change in Initial Position 17
5.3.2 Change in Initial Velocity 18

6 Discussion **19**
6.1 Numerical Stability and Energy Conservation 19
6.2 Trajectory Simulation 19
6.3 Evidence of Chaos 19

7 Bibliography **20**

1 Abstract

This project presents a simulation of the motion of a particle around a dipole simulated using Python and certain numerical methods. The system is modeled with the help of physical laws such as Coulomb's Law and Newton's Second Law of Motion to derive a system of differential equations, which is then solved with numerical methods. A comparative analysis between Euler's method and Velocity Verlet integration was performed, confirming that the Verlet method is significantly more effective at conserving the total mechanical energy of the system.

Using Verlet's method multiple trajectories were simulated. In addition, the system was tested for chaotic properties by introducing microscopic changes of 10^{-6} to the initial position and velocity and checking for divergence of the trajectories. Log-distance separation plots confirmed that these minor changes lead to macroscopic divergence over time, demonstrating that the dipole environment is highly sensitive to initial conditions.

2 Introduction

Charged particles are the fundamental building blocks of the matter that surrounds us. What makes them special to other bodies is that they are also affected by electromagnetic forces. These electromagnetic forces consist of an electric component, created by charge, and a magnetic component, produced by the motion of that charge and the existence of a magnetic field. A certain way of an electric force acting on a charged particle is the particle being in the surroundings of a dipole. A dipole is a system that includes two fixed, equal and opposite charges separated by a fixed distance.

In order to compute the movement of a charged particle around a dipole, I will use a combination of Coulomb's law and Newton's second law of motion in order to receive a vector function of the particle's acceleration. I want to simulate the movement and velocity of the charged particle; therefore, I will want to solve this system of differential equations. However, this system has no general closed-form solution for arbitrary initial conditions and is therefore solved numerically. Using numerical methods, I can approximate the solution and describe the movement of the charged particle. A numerical method is a way of approximating a solution to a complex mathematical problem, which has no analytical solution.

In this project I will use two different numerical methods for solving differential equations, Euler's method and Verlet integration. With the help of these methods, I can investigate 3 dimensional trajectories of charged particles around a dipole with varying properties such as mass, amplitude and sign of charge, starting position relative to the dipole and starting velocity. I will research the stability of the two numerical methods, simulate special trajectories and check if the system is chaotic (sensitive to starting properties).

3 Theoretical Basis

3.1 Governing Physical Laws

3.1.1 Particle Movement

To understand how a charged particle moves in this simulation, we must look at the basic laws of physics that govern its behavior. The most important factors are the electrostatic forces created by the two fixed poles and the fundamental laws of motion that determine how the particle responds to those forces. By calculating these interactions, we can predict the particle's path and track how its total energy changes over time.

By Coulomb's Law each side of the dipole puts a force on the moving particle:

$$\vec{F_e} = k\frac{q_1 q_2}{|r|^3}\vec{r}$$

Which has the following size:

$$|F_e| = k \cdot \frac{|q_1 q_2|}{r^2}$$

Where k is the Coulomb constant ($k = 8.99 \cdot 10^9 N \cdot m^2/C^2$), q_1, q_2 are the charges of the particles acting an electric force on each other and r is the distance between the particles. The direction of the force depends on the sign of the charge of the two particles. If they have opposite charge signs (one positive and one negative) then they attract each other meaning the direction is towards each other. Otherwise, (both positive or both negative) then they repel each other. In my system, the only forces acting are the forces that the dipole acts on the moving particle even though in theory the moving particle should also act a force on the particles that make up the dipole. Therefore, we can use Newton's second law of motion to receive the following:

$$\vec{F} = m_p \vec{a}_p$$

$$\vec{a}_p = \frac{\vec{F}_{pos} + \vec{F}_{neg}}{m_p}$$

Where $\vec{F}_{pos}, \vec{F}_{neg}$ are the forces the positive and negative particles of the dipole act respectively on the particle, which are calculated by Coulomb's Law as shown above.

As I am simulating the movement in 3 dimensions we can split this into 6 first order differential equations. We can write the following equations for position and velocity:

$$\frac{dx}{dt} = v_x, \frac{dv_x}{dt} = \frac{F_{pos_x} + F_{neg_x}}{m_p}$$

We can write the same for every axis (x, y, z) and receive 6 first order differential equations. Based on this system of first order differential equations we can use numerical methods to model the particle's movement.

3.1.2 Conservation of Energy

Energy is the capacity of a system to perform work. In classical mechanics, a fundamental law states that in a closed system governed by conservative forces (forces where the total work done on a particle depends only on its starting and ending points), the total mechanical energy remains constant over time. In this system, the moving particle possesses two forms of energy: Kinetic Energy (K), related to its motion, and Electrostatic Potential Energy (U), related to its position within the dipole's field. The formulae for calculating these are:

$$K = \frac{mv^2}{2}, U = \frac{kqQ}{R}$$

$$E_{total} = K + U$$

In a closed system where only conservative forces (such as the electrostatic force) are present, the total mechanical energy remains invariant. This implies that the sum of kinetic and potential energy is constant, regardless of the path the particle takes between any two points in time, t_1 and t_2:

$$E_{\text{total}}(t_1) = E_{\text{total}}(t_2)$$

Therefore:

$$\Delta E = E_{\text{total}}(t_1) - E_{\text{total}}(t_2) = 0$$

In our simulation we will want the change in energy to be as small as possible in order for it to be as accurate as possible.

3.2 Numerical Methods

The two numerical methods I will use are Euler's method and Verlet's Velocity integration method.

3.2.1 Mathematical Theory

These numerical methods are based on the one of the most important principles in calculus: the differential. The differential is defined as the linear approximation of a function based on a small step dt and the derivative at that point. The mathematical formula for the approximation with the differential is:

$$f(t_1) \approx f(t_0) + dt \cdot f'(t_0)$$

The numerical method use this formula in order to approximate a solution of a first order differential equation in the form:

$$\frac{dx}{dt} = f(x, t)$$

Meaning at any given x_0, t_0 we can calculate the derivative at that point and after choosing a step size (dx) we can approximate the next t value we want to calculate t_1.

3.2.2 Euler's Method

Euler's numerical method given a differential equation in the above form and a starting x_0, t_0 approximates a list of values $[x_1, x_2, ..., x_n]$ at points $[t_1, t_2, ..., t_n]$ where $t_i = t_0 + i \cdot dt$. Below appears the pseudocode of the simple case of Euler's numerical method:

Algorithm 1 Basic Euler's Method

```
procedure EULER(x_0, t_0, Δt, steps, f)
    x ← x_0                                  ▷ Initial value of the dependent variable
    t ← t_0                                  ▷ Initial value of the independent variable
    for i = 1 to steps do
        f_val ← f(x, t)                      ▷ Calculate the slope dx/dt
        x ← x + f_val · Δt                   ▷ Update step: x_{n+1} = x_n + f(x_n, t_n)Δt
        t ← t + Δt                           ▷ Increment time
    end for
    return x
end procedure
```

Algorithm 2 The Velocity Verlet Integration Method

```
procedure VelocityVerlet(x_0, v_0, Δt, steps, f)
    x ← x_0
    v ← v_0
    for i = 1 to N do
        a_current ← a(x)                          ▷ Calculate acceleration at current position
        v_half ← v + 0.5 · a_current · Δt         ▷ Velocity half-step
        x ← x + v_half · Δt                       ▷ Full position update
        a_next ← a(x)                             ▷ Calculate acceleration at new position
        v ← v_half + 0.5 · a_next · Δt            ▷ Finish velocity update
    end for
    return x, v
end procedure
```

3.2.3 Verlet's Method

Verlet's Velocity method is slightly different to Euler's method. It is a method for numerically solving second order differential equations, meaning that the acceleration is given as a function. This method fits more for physical systems than Euler's method for some reasons that we will talk about later in the article. Verlet's method receives a second order differential equation and starting position and velocity. The method updates the position and velocity in the following way shown in the algorithm below:
As can be seen in the above pseudocode the algorithm updates the velocity based firstly by a "half-step" then updates the position based on this "half velocity". After this, the algorithm recalculates the acceleration at the new position and finally updates the velocity to the full step.

3.2.4 Conservation of Energy in Numerical Methods

As described previously, in order for the approximations to be physically accurate the simulation will aim for the change in energy to be as small as possible. To check this change in energy, I will calculate the energy at the start and end of the particle's movement and hence calculate the change of energy. Later I will compare the two numerical methods and see which one is more accurate according to the principle of conservation of energy.

3.3 Chaos Theory

The scientific study of systems that are highly sensitive to initial conditions is called **Chaos Theory**. While some of these systems are governed by deterministic laws, meaning their future behavior is entirely determined by their initial state with no random elements involved, they appear to behave in a random and unpredictable manner over time. Systems with these properties are called chaotic.

This phenomenon is famously illustrated by the "Butterfly Effect," where a microscopic change in the starting parameters of a system can lead to vast divergences in the resulting trajectory. An example of a chaotic system in the field of physics is the double pendulum. A pendulum which is connected to another pendulum creates great randomness and chaos in its movements even though all the forces can be calculated directly.

4 Methodology

4.1 Development Environment and Software Stack

Throughout this project I coded with VSCode and utilized Git with GitHub for version control (project in the following repository: https://github.com/t0merez/Particle-Motion-Simulation). I coded with Jupyter Notebooks (.ipynb files) with a Python kernel. I stored all my relevant functions in a separate Python file and imported them into my Jupyter Notebook.

Throughout the coding of the project I used multiple Python libraries for storing data, simulations and more such as:

NumPy: Used for vectorized 3D math and linear algebra, essential for the acceleration and position updates in the Verlet loop.

Matplotlib: Facilitated the generation of static 3D trajectory plots and distance-over-time graphs.

Plotly Express: Provided interactive 3D visualizations and animations to better observe the particle's behavior.

Pandas: Enabled efficient data management and the precise calculation of energy drift across thousands of simulation steps.

Copy: Used to generate identical particle instances for the divergence tests, ensuring independent trajectory calculations.

4.2 System Configuration

4.2.1 The Electrostatic Dipole Environment

As described earlier and shown in ***(Dipole Figure)*** my system includes a dipole and a particle in its surroundings in a 3D space. The particle is affected by the electrical force the two sides of the dipole create. In my Python file containing all functions I defined the two sides of the dipole as following:

Table 1: Physical Properties of the Simulation System

Entity	**Position** (x, y, z)	**Charge** (q)	**Mass** (m)
Positive Pole	$(0, 0, 1)$	$+1.0$ C	∞ (Fixed)
Negative Pole	$(0, 0, -1)$	-1.0 C	∞ (Fixed)

4.2.2 Initial Particle Properties

For every simulation, I defined a particle with properties such as above and also a starting velocity. I assumed that the particles are released from rest meaning they have no starting acceleration. For all particles I defined them to have mass of 1.0 and a charge size of 1.0 with differing charge signs. I chose the particle's starting position and speed based on physical intuitive for what properties would make special trajectories.

It should be noted that the physical values used in the simulation (charges of ± 1 C, distances of order 1 m and masses of 1kg) are not realistic for microscopic particle systems. These values are chosen for numerical convenience, effectively working in normalized units to better visualize the particle dynamics.

4.3 Code Structure and Implementation

4.3.1 Particle Class

The particles have many properties therefore with the help of Object Oriented Programming in Python I created a class called Particle which will hold all the data needed about a certain particle.

Listing 1 Particle Class Definition.

```
class Particle:
    def __init__(self, position, velocity, mass, charge):
        self.position = np.array(position)
        self.velocity = np.array(velocity)
        self.mass = mass
        self.charge = charge
        self.acceleration = np.zeros(3)
```

4.3.2 Acceleration Calculation

According to the equations shown in ***(Final Equations for Simulation)*** and with the help of NumPy functions I calculate the vector of acceleration of a particle in the following way:

Listing 2 Acceleration Calculation.

```
def compute_force(particle):
    r_pos = particle.position - positive_side.position
    r_neg = particle.position - negative_side.position
    # Calculate the magnitude of the forces
    F_pos = K * particle.charge * positive_side.charge / np.linalg.norm(r_pos)**2
    F_neg = K * particle.charge * negative_side.charge / np.linalg.norm(r_neg)**2
    # Calculate the direction of the forces
    F_pos_vector = F_pos * r_pos / np.linalg.norm(r_pos)
    F_neg_vector = F_neg * r_neg / np.linalg.norm(r_neg)

    # Total force is the sum of the forces from both poles
    total_force = F_pos_vector + F_neg_vector

    return total_force

# Calculate the acceleration of the particle based on the total force
def calculate_acceleration(particle):
    total_force = compute_force(particle)
    particle.acceleration = total_force / particle.mass
```

4.3.3 Implementation of Euler's Method

The algorithm at ***(Basic Euler's Method)*** showcases the general idea in Euler's numerical method. However in my simulation there is not one differential equation as is here and not one variable but 2 differential equations with 3 dimensions meaning 6 differential equations. In my code I defined a particle object which stores all the charges properties: mass, charge, position vector and velocity vector. Firstly I calculate the acceleration of the charge then I update the next step's velocity based on Euler's method. After that, with the new velocity I update the new position also based on Euler's method. Below is the code:

Listing 3 Euler's Numerical Method for Simulation.

```python
def simulate_Euler(p, dt=1e-7, steps=num_of_steps):
    pos = np.zeros((steps + 1, 3))
    vel = np.zeros((steps + 1, 3))
    # Store initial state
    pos[0] = p.position
    vel[0] = p.velocity
    for i in range(steps):
        calculate_acceleration(p)
        # Update velocity and position using Euler's method
        p.velocity = p.velocity + p.acceleration * dt
        p.position = p.position + p.velocity * dt

        pos[i+1] = p.position
        vel[i+1] = p.velocity

    return pos, vel
```

As can be seen my method returns two lists of the position and the velocity for every step because in the end I will want to do a simulation of the movement not just see where the particle ended up.

4.3.4 Implementation of Velocity Verlet Integration Method

My implementation of Velocity Verlet's method is nearly identical to the algorithm shown in ***(Velocity Verlet Integration Method)*** but I store the positions and velocities in a list instead of just returning the final position and velocity.

Listing 4 Velocity Verlet's Method for Simulation.

```python
def simulate_Verlet(p, dt=1e-7, steps = num_of_steps):
    pos = np.zeros((steps + 1, 3))
    vel = np.zeros((steps + 1, 3))
    # Store initial state
    pos[0] = p.position
    vel[0] = p.velocity
    for i in range(steps):
        calculate_acceleration(p)
        a_current = p.acceleration
        # Update velocity by half-step
        v_half = p.velocity + 0.5 * dt * a_current
        # Update position by full-step
        p.position = p.position + dt * v_half
        # Get new acceleration at the new position
        calculate_acceleration(p)
        a_new = p.acceleration
        # Finish velocity update
        p.velocity = v_half + 0.5 * dt * a_new
        pos[i+1] = p.position
        vel[i+1] = p.velocity
    return pos, vel
```

4.3.5 Parameter Choices

For the simulation of the movements using the numerical methods shown above the value of the size of the step, dt, is very important. If I chose too large of a dt value, I would receive that the particle will jump too much each step and then the movement will not be accurate. On the other hand, if I chose a very small dt value the computation time to receive a simulation that portrays interesting trajectories will be very large. Therefore, in order to keep a trade-off of accuracy and computation time I chose $dt = 10^{-7}$. This value keeps the trajectory accurate and also does not take too much time to compute. Combined with $25,000$ steps the simulation calculates 25 microseconds (0.000025 seconds) of the particle's trajectory with great accuracy and very quickly. Even though this 25 microseconds sounds very little it is enough to clearly see the particle's trajectories as the particle has very high velocity and acceleration.

4.4 Simulating Trajectories of Particles

4.4.1 Simulations of Cases

I defined 4 different particles with varying starting conditions and simulated their movements using Verlet's method for reasons I will describe later on. I defined particles which have the following trajectories relative to the dipole:

Stable Orbit: A particle which orbits around the dipole but not on the same axis all the time.

Figure 8: A particle that moves like a figure 8 around the dipole.

Scattering Particle: A particle that moves in an unpredictable manner and scatters around the dipole.

Circular Particle: A particle that moves in a perfect circular motion around the dipole. It does roughly two thirds of a circle in its motion.

4.4.2 Comparison of Energy Conservation between Numerical Methods

For each trajectory simulation described above I calculated the change in energy between the start and end points. I did this calculation for each numerical method and compared.

4.4.3 Investigating Chaotic Properties of the System

For multiple cases, I defined two particles which differ only in their starting position or starting velocity but only by very small values. For these two particles which differ slightly I plotted a log-distance over time graph and also plotted their trajectories on the same 3D graph. With the help of these two graphs I can conclude if the system is chaotic.

5 Results

5.1 Trajectory Simulations

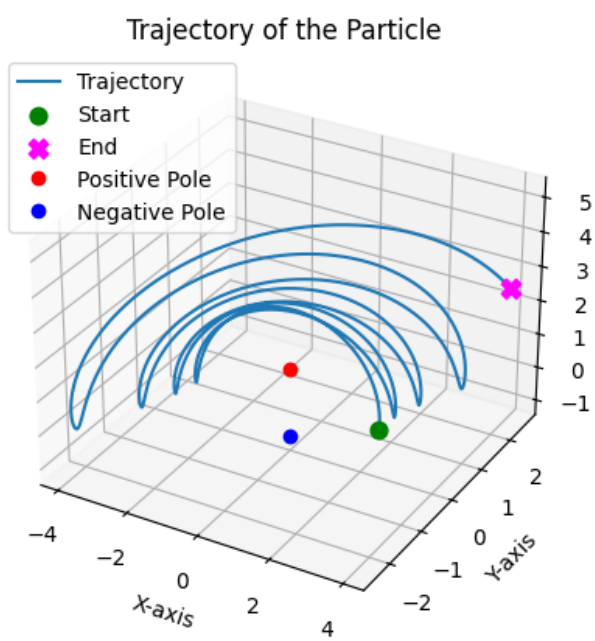
Below I will describe the particles I defined and present their interesting trajectories. All of these simulations were simulated with Verlet's integration numerical method. For all simulation the positive side of the dipole is red and the negative side is the blue. In addition, the starting point is green and the ending point is pink where the blue line is the trajectory that connects them.

5.1.1 Stable Orbit

This is the case of a negatively charged particle in a static dipole field. The particle exhibits a complex but bounded orbital pattern, oscillating between the positive (red) and negative (blue) poles.

Table 2: Initial State Configuration for Particle Trajectory

Property	**Initial Value**
Position $\mathbf{r}_0$	$(1.5, 0.0, 0.5)$
Velocity $\mathbf{v}_0$	$(0.0, 2.0, 0.0)$
Charge q	-1.0 C
Mass m	1.0 kg

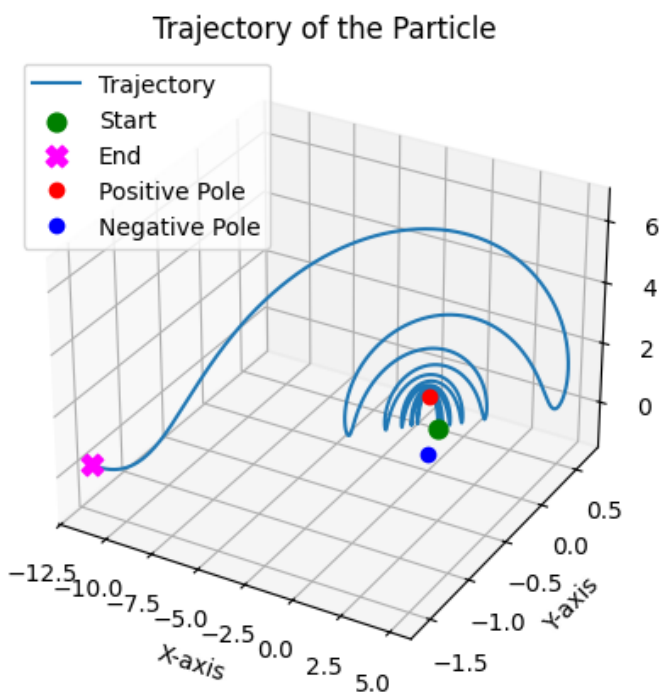


5.1.2 Sling Shot

This case demonstrates a high-gradient interaction. The particle enters a tight orbital loop near the dipole center before being ejected from the dipole.

Table 3: Initial State Configuration for Case 2

Property	**Initial Value**
Position $\mathbf{r}_0$	$(0.5, 0.0, 0.0)$
Velocity $\mathbf{v}_0$	$(0.1, 0.0, 0.1)$
Charge q	-1.0 C
Mass m	1.0 kg

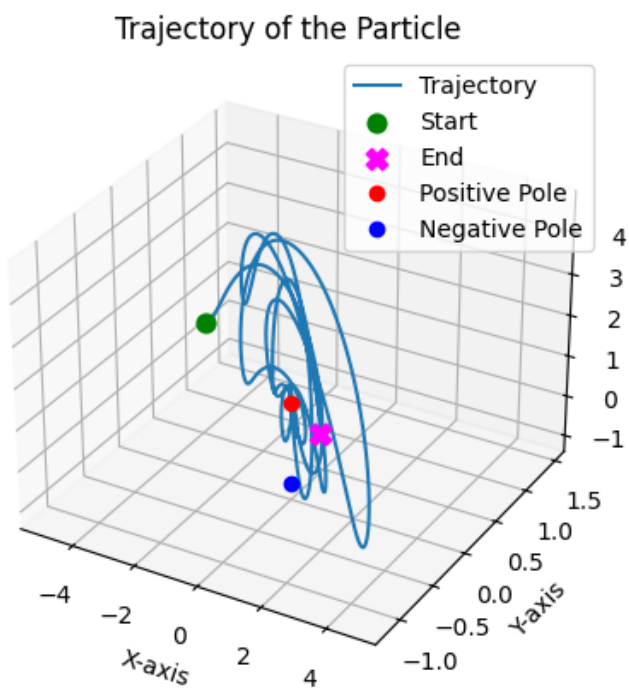


5.1.3 Scattering Movement

In this case the particle exhibits a complex, non-repeating pattern, oscillating unpredictably between the positive and negative poles.

Table 4: Initial State Configuration for Case 3

Property	**Initial Value**
Position $\mathbf{r}_0$	$(-2.0, 1.0, 1.5)$
Velocity $\mathbf{v}_0$	$(1.2, -0.5, -0.8)$
Charge q	-1.0 C
Mass m	1.0 kg

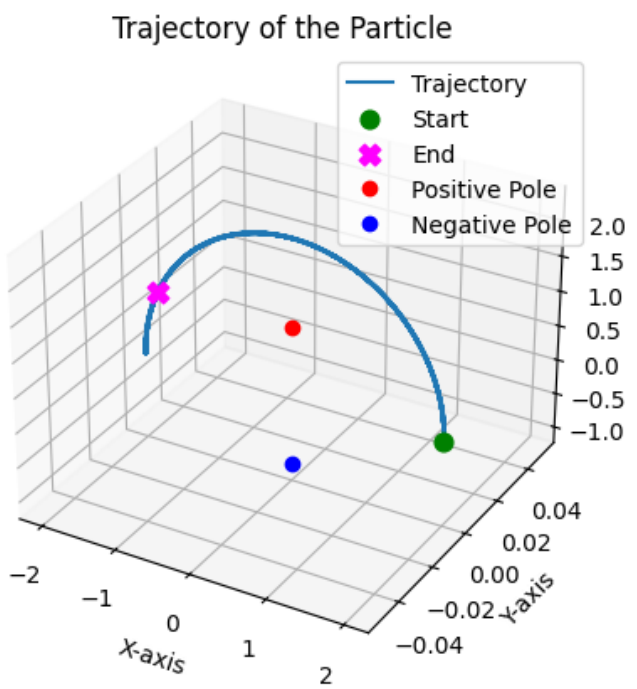


5.1.4 Circular Motion

With no initial Y-velocity, the particle oscillates in the XZ-plane, tracing a predictable arc governed by the vertical potential gradient.

Table 5: Initial State Configuration for Case 4

Property	**Initial Value**
Position $\mathbf{r}_0$	$(2.0, 0.0, -1.0)$
Velocity $\mathbf{v}_0$	$(-1.5, 0.0, 1.5)$
Charge q	-1.0 C
Mass m	1.0 kg



5.2 Energy Conservation of Numerical Methods

For each simulation above I calculated the energy change for both numerical methods and received the following table:

Table 6: Comparison of Energy Conservation (ΔE) between Numerical Methods

Case	**Motion Type**	**Euler ΔE (J)**	**Verlet ΔE (J)**
1	Stable Orbit	-1.4800	-0.0064
2	Sling Shot	-3.1210	-0.0091
3	Scattering Motion	-2.8540	-0.0072
4	Circular Motion	-1.1520	-0.0041

As can be clearly seen in every case the change in energy using Verlet's method is much smaller than Euler's method. This means that Verlet's method abides by the principle of conservation of energy much better than Euler's method therefore it is also much more physically accurate.

5.3 Chaotic Properties

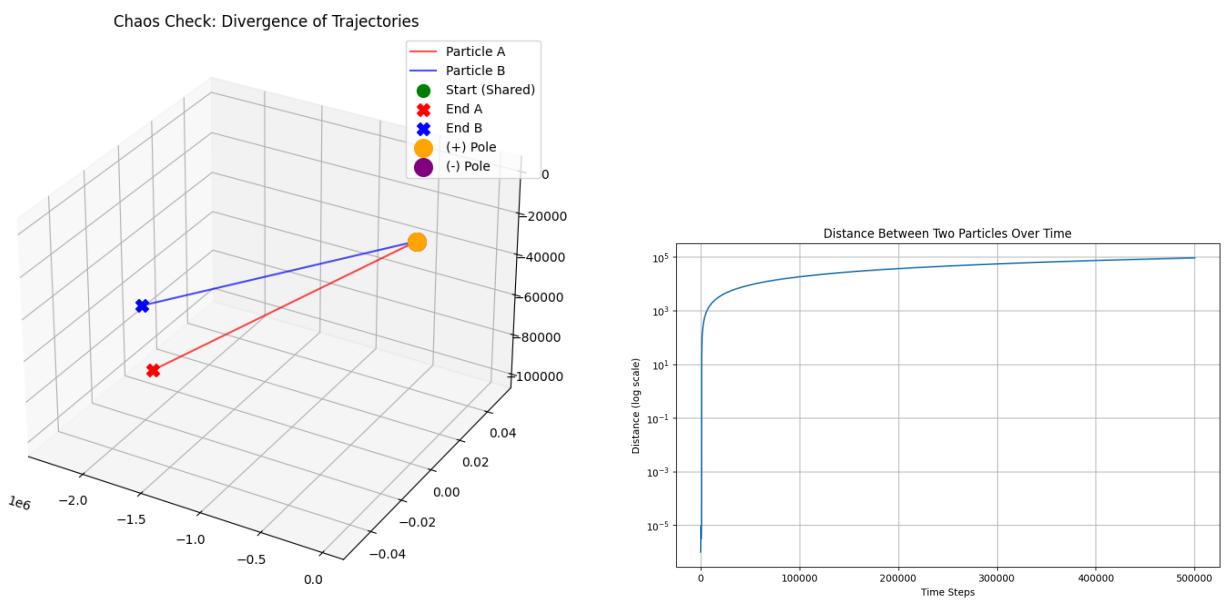
In this section of the project I checked the chaotic properties of the system. I will present two cases which show the chaos of the system with two figures: a log-distance over time graph between two particles with microscopic changes in initial conditions and a simulation of both particles on the same axis.

5.3.1 Change in Initial Position

This case shows the chaotic properties of the system where the only difference between the particles is in the initial position.

Table 7: Initial State Configuration for Initial Change in Position

Property	**Particle 1**	**Particle 2**
Position $\mathbf{r}_0$	$(1.5, 0.0, 0.5)$	$(1.5 + 10^{-6}, 0.0, 0.5)$
Velocity $\mathbf{v}_0$	$(0.0, 0.0, 50.0)$	$(0.0, 0.0, 50.0)$
Charge q	-1.0 C	-1.0 C
Mass m	1.0 kg	1 kg



(a) 3D Divergence of Trajectories

(b) Log-Distance Separation

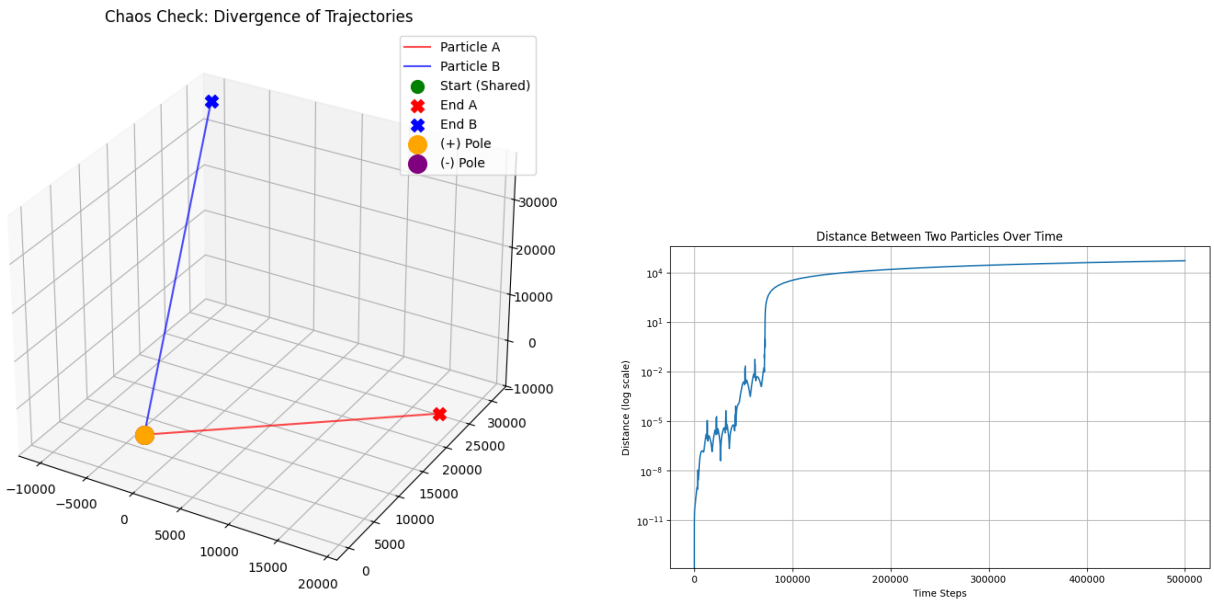
Figure 2: A microscopic position change of 10^{-6} leads to macroscopic separation, confirming deterministic chaos.

5.3.2 Change in Initial Velocity

This case presents the chaotic properties of the system and its sensitivity to initial conditions but not just one initial condition but other initial conditions such as velocity.

Table 8: Initial State Configuration for Initial Change in Velocity

Property	**Particle 1**	**Particle 2**
Position $\mathbf{r}_0$	$(-5.0, 1.0, 0.5)$	$(-5.0, 1.0, 0.5)$
Velocity $\mathbf{v}_0$	$(3500.0, 0.0, 0.0)$	$(3500.0 + 10^{-6}, 0.0, 0.0)$
Charge q	-1.0 C	-1.0 C
Mass m	1.0 kg	1 kg



(a) 3D Divergence of Trajectories

(b) Log-Distance Separation

Figure 3: A microscopic velocity change of 10^{-6} leads to macroscopic separation, confirming deterministic chaos.

6 Discussion

The results of this simulation provide a comprehensive look at the dynamics of a charged particle in a static dipole field. By analyzing the four distinct cases and the chaos check, several key conclusions can be drawn regarding both the physics of the system and the computational methods used to model it.

6.1 Numerical Stability and Energy Conservation

A primary goal in this project was to compare Verlet's integration method and Euler's numerical method in physical systems. As shown in **5.2** Verlet's method conserves energy accurately however Euler's method does not conserve energy. This confirms that Euler's method, while computationally simple, is unsuitable for long-term physical simulations due to its lack of energy conservation and explains why I used Verlet's integration method for the simulations.

6.2 Trajectory Simulation

The successful simulation of diverse trajectories serves as a powerful validation of the numerical framework. By combining Coulomb's Law with Newton's Second Law, the software environment successfully replicated high-stakes physical interactions.

The fact that the simulation could consistently produce both bounded orbits and open escape paths—using the same underlying vector math proves that the implementation is physically correct. Furthermore, the circular motion test (**5.1.4**) acted as a critical sanity check; the code correctly handled the absence of Y-axis momentum, maintaining perfect 2D symmetry within a 3D computational space.

6.3 Evidence of Chaos

The results in **5.3** clearly show the chaotic properties of the dipole system. By introducing a microscopic change of 10^{-6} to an initial condition, the system eventually produced a macroscopic divergence in the particle's final state.

The log-distance plot confirms that the error between the two particles grows non-linearly, a hallmark of Deterministic Chaos. This confirms that while the physics is governed by fixed, non-random laws, the system's inherent sensitivity makes long-term prediction impossible without near-infinite precision.

7 Bibliography

References

[1] The Editors of Encyclopaedia Britannica. *Chaos theory*. Encyclopaedia Britannica, 2024. Available at: https://www.britannica.com/science/chaos-theory

[2] Wikipedia contributors. *Verlet integration*. Wikipedia, The Free Encyclopedia, 2026. Available at: https://en.wikipedia.org/wiki/Verlet_integration.

[3] Erik. *Verlet Integration*. MIT Fab Lab, NMM 2023. Available at: https://fab.cba.mit.edu/classes/864.23/people/Erik/verlet-integration/.

[4] The Editors of Encyclopaedia Britannica. *Coulomb's Law*. Encyclopaedia Britannica, 2024. Available at: https://www.britannica.com/science/Coulombs-law.

[5] The Editors of Encyclopaedia Britannica. *Newton's laws of motion*. Encyclopaedia Britannica, 2025. Available at: https://www.britannica.com/science/Newtons-laws-of-motion.

[6] Plotly Technologies Inc. *Collaborative data science*. Montreal, QC, 2025. Available at: https://plot.ly.

[7] Harris, C.R., Millman, K.J., van der Walt, S.J. et al. *Array programming with NumPy*. Nature 585, 357–362, 2020. Available at: https://numpy.org.

[8] Rozenfield, S. *Computational Science: Assignments and Reference Implementations*. Hemda, Tel Aviv, 2026. Internal course materials and personal communication.